



Writing Scientific Reports in Computer Science and Engineering

A Practical Guide and L^AT_EX Template for Bachelor Students

Course: *DTE-2602 Introduction to Machine Learning and Artificial Intelligence*

Instructor: *Roy Espen Olsen*

August 21, 2026

Abstract

This document has two purposes at once. First, it is a short course on how to plan, structure, and write a scientific report in computer science(CS) and engineering, aimed at bachelor students writing their first lab reports, project reports, or thesis chapters. Second, the document *is itself* written in L^AT_EX and is meant to be copied and reused as a starting template: every section demonstrates the very techniques it describes — figures, tables, algorithms, code listings, equations, and citations are all shown “in the wild” so that students can see the source and the rendered result side by side. Read Section 2 for the expected structure of a report, and use Section 9 as a final self-review checklist before submission.

Contents

1	Introduction	3
2	Structure of a Report	4
2.1	Abstract	4
2.2	Introduction vs. Background	4
2.3	Results vs. Discussion	5
3	Writing Style	6
3.1	Clarity over cleverness	6
3.2	Tense	6
3.3	Passive vs. active voice, and first person	6
3.4	Precision and hedging	6
3.5	Consistency	7
3.6	Paragraph and section structure	7
4	Figures and Tables	8
4.1	Tables	8
4.2	Placement and referencing	9
5	Algorithms and Code	10
5.1	Pseudocode	10
5.2	Source code listings	10
5.3	Equations	11
6	Citations and References	12
6.1	Why and when to cite	12
6.2	Using <code>natbib</code> and BibTeX	12
6.3	Citing software, datasets, and AI tools	13
7	Practical $\LaTeX$ Tips	14
8	Common Mistakes We See Every Year	15
9	Pre-submission Checklist	16
	References	17

1 Introduction

A scientific report is not a diary entry, a sales pitch, or a code comment expanded to ten pages. It is a structured argument: you had a question or a problem, you did something about it in a principled way, and you obtained results that a professor, teacher or even a stranger — your reader — can understand, judge, and in principle reproduce. Everything in this guide follows from that one sentence.

In computer science and engineering, this means your report typically documents one of the following:

- an **experiment** (e.g. benchmarking two algorithms, evaluating a machine-learning model),
- a **system** you designed and built (e.g. a compiler, a web service, an embedded controller),
or
- an **investigation** of an existing system or dataset.

In all three cases, the reader — your instructor, a fellow student, or your future self in six months — should be able to answer three questions after reading your report: *What did you do? Why did you do it that way? What did you find, and what does it mean?*

Tip

Write for a reader who is intelligent and technically competent, but who was *not* in the room when you did the work. They do not know what you meant to say — only what you actually wrote.

This document walks through the expected structure of a report (Section 2), how to write clearly and in an appropriate academic style (Section 3), how to present figures, tables, algorithms and code (Sections 4 and 5), how to cite sources correctly (Section 6), a few L^AT_EX-specific tips that will make your life easier (Section 7), a list of mistakes we see every year (Section 8), and a checklist to run through before you submit (Section 9).

2 Structure of a Report

Most scientific writing in engineering follows a variant of the **IMRaD** pattern: *Introduction, Methods, Results, and Discussion*. Table 1 shows how this maps onto a typical CS/engineering report and what each part is *for* — not just what it is called.

Table 1: Typical section structure for a CS/engineering report and the question each section must answer.

Section	Question it must answer
Title	What is this, in one line?
Abstract	In 150–250 words: problem, approach, key result.
Introduction	Why does this problem matter, and what exactly did you do about it?
Background / Related Work	What does the reader need to know, and how does your work relate to existing solutions?
Design / Methodology	How does your system or experiment work, and why did you make these design choices?
Implementation	What did you actually build? (Only if relevant — keep it high-level; code goes in an appendix or repository.)
Evaluation / Results	What did you measure, and what did you observe? (Facts only, no interpretation yet.)
Discussion	What do the results <i>mean</i> ? What are the limitations?
Conclusion	What is the one-paragraph takeaway, and what would you do next?
References	Who else’s work did you build on or compare against?

2.1 Abstract

The abstract is the most-read and least-understood part of a report. It is *not* an introduction and it is *not* a table of contents in prose form. A good abstract compresses the whole report into four moves: (1) one sentence of context/problem, (2) one sentence of what you did, (3) one or two sentences of what you found, (4) optionally, one sentence on why it matters. Write the abstract *last*, after the rest of the report is stable.

2.2 Introduction vs. Background

Students often merge these, which weakens both. The **introduction** motivates the problem and states your contribution in the first page — a reader who stops after the introduction should still know what you did and why. The **background/related work** section is where you teach the reader the prerequisites and position your work relative to existing approaches. If a fact is needed to understand your method, it belongs in background, not scattered later.

Tip

A useful test: could you swap the introduction of two unrelated reports and have both still make sense? If yes, your introduction is too generic — it needs to say what *your* project specifically does.

2.3 Results vs. Discussion

Keep these separate, even if only conceptually. **Results** report what happened, ideally with a table or figure and minimal commentary. **Discussion** is where you interpret: Was this expected? Does it support your hypothesis? What are the threats to validity (measurement noise, small sample, unrepresentative test data, hardware differences)? Mixing the two makes it hard for a reader — or a grader — to tell which sentences are evidence and which are your opinion.

Exercise

Take the last report you wrote (for this course or another) and highlight every sentence in your results section that contains a judgement word (“surprisingly”, “unfortunately”, “clearly better”). Each one is arguably discussion, not results. How many did you find?

3 Writing Style

The primary goal of technical writing is to communicate ideas precisely and efficiently. In a CS or engineering report, clear language is more valuable than sophisticated-sounding language. The following guidelines cover the main aspects of technical style: clarity, tense, voice, precision, terminology, and paragraph structure.

3.1 Clarity over cleverness

Prefer the plain word to the fancy one, and the short sentence to the long one. “We used” beats “We made utilization of”. “The algorithm runs in $O(n \log n)$ time” beats “The algorithm exhibits logarithmic scaling behaviour with respect to input cardinality”. If a sentence takes two readings to parse, split it.

3.2 Tense

As a simple default:

- **Past tense** for what you did and what you observed: “We evaluated three schedulers”, “The cache hit rate dropped to 62%”.
- **Present tense** for facts that remain true and for describing the content of your document itself: “Figure 1 shows the results”, “TCP guarantees in-order delivery”.

3.3 Passive vs. active voice, and first person

Both active and passive voice are acceptable in engineering writing; the goal is clarity, not a blanket rule. Active voice is usually more direct: “We measured latency under three loads” is easier to parse than “Latency was measured under three loads by us”. Passive voice is useful when the actor is irrelevant or obvious: “The packets were captured using `tcpdump`” is fine because *who* ran the tool matters less than *how* the data was captured. Using “we” for actions your team actually performed is standard in modern CS writing — check your course or venue’s convention if unsure.

Common pitfall

Do not mix an impersonal, passive style in the results with a chatty “I think” style in the discussion. Pick a register and keep it consistent across the whole report.

3.4 Precision and hedging

Say exactly what you mean and no more. “The system is fast” is not a claim you can defend; “the system processes 12,400 requests/s at the 95th-percentile latency of 8ms on the hardware described in the experimental setup section” is. Conversely, do not overclaim: if you tested on one dataset, say so, and do not generalise to “all workloads”. Words like “clearly”, “obviously”, and “trivially” rarely help the reader and often hide a step you skipped.

3.5 Consistency

Pick one term for each concept and stick to it. If your system is a “scheduler” in Section 2, do not call it a “dispatcher” in the evaluation unless you have explicitly defined them as synonyms. The same applies to notation: if n is the number of nodes in Section 5, it must still be the number of nodes on the last page.

3.6 Paragraph and section structure

One idea per paragraph, and put the main point in the first sentence (a “topic sentence”). Readers — and graders — skim; a report whose first sentence of every paragraph forms a coherent summary is a well-structured report. Use subsections to help navigation, but avoid sections with a single sentence and no subsections around them; if a subsection is that short, it is probably a paragraph, not a subsection.

Tip

Read your own paragraph’s first sentence out loud, in order, skipping everything else. If that summary doesn’t make sense as an outline of your argument, restructure the paragraphs.

4 Figures and Tables

Figures and tables carry evidence; captions carry meaning. A common mistake is a one-line caption like “Results” under a plot with three unlabelled axes. Figure 1 shows a better pattern: the caption states what is plotted, the axes, and the one-sentence takeaway, so the figure is understandable even if a reader skips the surrounding text.

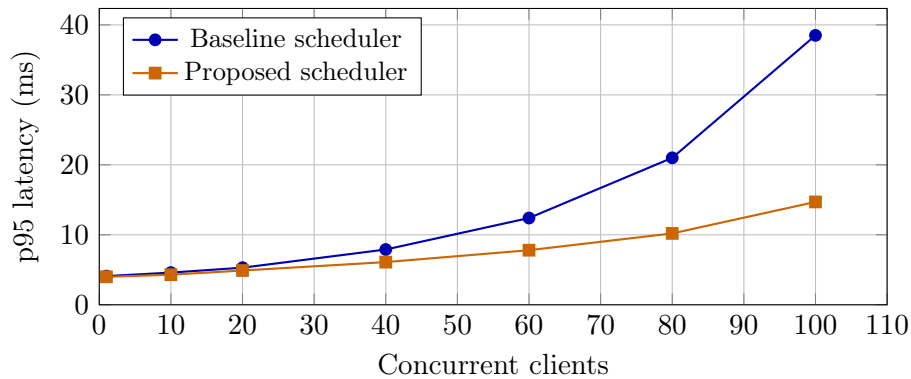


Figure 1: 95th-percentile request latency versus number of concurrent clients, for the baseline round-robin scheduler and the proposed load-aware scheduler (Section 5). The proposed scheduler degrades more gracefully under load. (*Illustrative numbers, not real measurements — in your own report this caption should also point to the section describing your experimental setup.*)

Tip

Every figure and table needs: a caption that stands on its own, labelled axes with units, a legend if there is more than one series, and a reference from the body text (“`\cref{fig:latency}` shows...”). A figure the text never mentions is a figure the reader will skip.

4.1 Tables

Use tables for precise numbers the reader may want to look up; use figures for trends and shapes. Table 2 is a minimal example using the `booktabs` package, which gives cleaner horizontal rules than the default $\text{\LaTeX}$ table style (avoid vertical rules and “double” borders — they add visual noise, not information).

Table 2: Comparison of the two schedulers at 100 concurrent clients. Throughput in requests per second; latency in milliseconds.

Scheduler	Throughput (req/s)	p50 latency	p95 latency
Baseline (round-robin)	8,120	9.8 ms	38.5 ms
Proposed (load-aware)	11,940	6.1 ms	14.7 ms

Common pitfall

Do not screenshot a table from a spreadsheet or a terminal. Rebuild it natively in $\text{\LaTeX}$ (as above) so that fonts, alignment, and decimal points match the rest of the document.

4.2 Placement and referencing

Let $\text{\LaTeX}$ place floats (`htbp`) rather than fighting it with exact positions; refer to figures/tables by label (`\cref{fig:latency}`) rather than by position (“the figure above”), since positions shift as the document is edited.

5 Algorithms and Code

Algorithms and implementation details should be presented at the level needed for the reader to understand, reproduce, or evaluate the work. Use pseudocode when the logic of an algorithm is the important part, source code listings when a specific implementation detail matters, and equations when the underlying computation or relationship needs to be stated precisely. Avoid reproducing implementation details that do not contribute to the argument; the complete source code can instead be provided in an appendix or repository.

5.1 Pseudocode

When the *logic* of a method matters more than a specific programming language, present it as pseudocode with the `algorithm2e` package, as in Algorithm 1. Pseudocode should be language-agnostic, use meaningful variable names, and be short enough to read in under a minute. If it needs more than about 20 lines, summarise the outer loop in prose and put the full version in an appendix.

Algorithm 1: K-means clustering

Input: Dataset $X = \{x_1, \dots, x_n\}$, number of clusters k

Output: Cluster assignments $c_1, \dots, c_n$ and centroids $\mu_1, \dots, \mu_k$

1 Initialise k centroids $\mu_1, \dots, \mu_k$ randomly

2 **repeat**

3 **foreach** $x_i \in X$ **do**

4 $c_i \leftarrow \arg \min_{j \in \{1, \dots, k\}} \|x_i - \mu_j\|^2$

5 **foreach** $j \in \{1, \dots, k\}$ **do**

6 $\mu_j \leftarrow \frac{1}{|\{i: c_i = j\}|} \sum_{i: c_i = j} x_i$

7 **until** *convergence*

8 **return** $c_1, \dots, c_n, \mu_1, \dots, \mu_k$

5.2 Source code listings

When the exact code matters (e.g. when explaining a preprocessing step or a model implementation detail), use the `listings` package rather than a screenshot, so the code is searchable, copy-pasteable, and consistently styled. Listing 1 shows a short Python implementation using `scikit-learn`.

```

1 from sklearn.cluster import KMeans
2
3 model = KMeans(n_clusters=3, random_state=42)
4 labels = model.fit_predict(X)
```

Listing 1: Clustering data points with K-means.

Tip

Only include code that illustrates a specific point you are making in the text. A report is not the place to reproduce your entire codebase — link to a repository or put long listings in an appendix.

Common pitfall

Do not paste an indented code block as plain text without syntax highlighting or numbering; it becomes very hard to reference (“see line 14”) and hard to read. Use `listings` or `minted` consistently throughout the report.

5.3 Equations

Number equations you refer to later, and treat the equation as part of the sentence grammatically. For example, the K-means objective can be defined as

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2, \quad (1)$$

where x_i is the i th data point, c_i is its assigned cluster, and μ_{c_i} is the centroid of that cluster. Equation (1) expresses the objective that K-means minimises: the total squared distance between each data point and the centroid of its assigned cluster. The equation therefore gives the reader a precise mathematical description of the algorithm presented in Algorithm 1.

6 Citations and References

Citations make clear which ideas, methods, results, and resources come from existing work and which are the author’s own. They allow readers to verify claims, locate the work on which the report builds, and distinguish established knowledge from new contributions. This section explains when citations are required, how to manage references with $\text{\LaTeX}$, and how to cite software, datasets, and AI tools appropriately.

6.1 Why and when to cite

Cite a source whenever you rely on a fact, method, figure, or idea that is not your own or not common knowledge in the field. This includes: background facts (e.g. the original definition of an algorithm [1]), tools and datasets you used, and prior work you compare against. A rule of thumb: if you could not have written the sentence without having read that source, cite it. Foundational results you build directly on should also be cited, even if they are famous — for example, referencing Shannon’s original formulation when discussing channel capacity [2].

Common pitfall

Citing a source is not the same as quoting it. Paraphrase in your own words and cite; use direct quotation marks only for the rare case where the exact wording matters (e.g. a definition from a standard). Copy-pasting text and adding a citation at the end of the paragraph is still plagiarism if the wording is not your own.

6.2 Using natbib and BibTeX

This template uses `natbib` with the `ieeetr` bibliography style, a numeric style common in CS/engineering venues. Entries live in `references.bib`; a minimal entry looks like:

```
1 @inproceedings{smith2023scheduling,  
2   author    = {Smith, Jane and Doe, John},  
3   title     = {Load-Aware Scheduling for Microservices},  
4   booktitle = {Proc. of ExampleConf},  
5   year      = {2023},  
6   pages     = {1--12},  
7 }
```

Listing 2: A BibTeX entry.

and is cited in the text with `\cite{smith2023scheduling}`, which renders as a bracketed number with the `ieeetr` style used here, e.g. the classic paper on machine intelligence [3]. (If you use an author–year style such as `plainnat` or `biblatex`’s `authoryear` instead, `natbib`’s `\citeta{...}` lets you use the author name as a noun in the sentence, e.g. “Turing `\citeta{turing1950}` asked whether machines can think.”) Every entry you cite must appear in `references.bib`, and (with numeric styles) every entry in the bibliography should be cited somewhere — do not pad the reference list.

Tip

If your own L^AT_EX install has `biblatex/biber` available (true on Overleaf and most modern TeX Live installs), you can swap this template's `natbib` block in `main.tex` for `\usepackage[backend=biber,style=ieee]{biblatex}` plus `\addbibresource{references.bib}` and `\printbibliography`; `biblatex` offers more citation styles and better internationalisation. Both approaches read the same `references.bib` file.

6.3 Citing software, datasets, and AI tools

Cite the tools and datasets that materially shaped your results, e.g. a specific library version, dataset, or pretrained model, the same way you would cite a paper — most have a recommended citation in their documentation or a Zenodo/DOI record. If you used a generative AI tool (e.g. for code suggestions or editing), follow your course's or institution's disclosure policy; when in doubt, state in a short paragraph (e.g. at the end of the introduction or in an acknowledgements section) what the tool was used for and confirm you verified the result yourself.

Tip

Keep an eye on citation style consistency: IEEE numeric ([1]), ACM, or author–year (APA/Chicago) — pick the one your course requires and use it throughout. Reference managers (Zotero, JabRef) export directly to `.bib` and will save you many hours over a semester.

7 Practical L^AT_EX Tips

This section is about the tool, not the writing — small habits that save time and prevent embarrassing errors.

- **One file per section.** As in this project (`sections/*.tex`, pulled in with `\input{}`), split long documents so you can navigate, diff, and collaborate more easily via version control.
- **Use `cleveref`.** Write `\cref{fig:latency}` instead of `Figure~\ref{fig:latency}`; it automatically inserts “Figure”, “Table”, “Section”, etc., consistently, and you only need to remember one command.
- **Never hand-number figures/sections/equations.** Always use `\label{}` and `\ref{}/\cref{}`. Manually typed numbers (“see Figure 3”) silently go stale the moment you add a figure earlier in the document.
- **Non-breaking spaces before references.** Write `Section~\ref{sec:x}` (with `~`) — `cleveref` handles this automatically, which is one more reason to use it.
- **Use version control.** Put the project in Git. L^AT_EX source is plain text and diffs beautifully; commit before major rewrites so you can always go back.
- **Compile often.** Do not write for an hour without compiling — errors are much easier to find one paragraph at a time than after 300 new lines.
- **`siunitx` for units and numbers.** For a report with many measurements, the `siunitx` package (e.g. `\SI{8.2}{ms}`) keeps units and number formatting consistent; not used in this template to keep dependencies minimal, but worth adding for data-heavy reports.
- **Spell- and grammar-check.** Use your editor’s checker or an external tool before submission; typos undermine an otherwise good report.
- **Keep the preamble organised.** As the document grows, move packages and custom commands into a separate `preamble.tex` file and load it from `main.tex` with `\input`. This keeps the main document focused on its content and makes the configuration easier to maintain.

Tip

Compile with `latexmk -pdf main.tex` (or the provided Makefile: run `make`) rather than calling `pdflatex` manually several times — `latexmk` re-runs `bibtex`/`pdflatex` exactly as many times as needed to resolve citations and cross-references.

8 Common Mistakes We See Every Year

The following are common issues that weaken otherwise solid reports; check for them before finalising your document.

1. **Abstract that only restates the title.** “This report describes our load balancer” tells the reader nothing an abstract should: what was the problem, what did you build, what did you find?
2. **No explicit research question or goal.** If a reader cannot point to one sentence in your introduction that says what you set out to answer or build, the rest of the report has nothing to be evaluated against.
3. **Results without a baseline or comparison.** “Our system achieves 95 % accuracy” means little without stating what a naive baseline achieves, or what prior work reports.
4. **Screenshots instead of native figures/tables.** Hard to read, inconsistent fonts, and impossible to cross-reference cleanly (see Section 4).
5. **Unlabelled axes or missing units.** A plot with axes called “x” and “y” communicates nothing.
6. **Wall-of-code appendices with no explanation.** Code should support the argument, not replace it; explain the interesting 10 lines rather than pasting 500.
7. **Inconsistent notation or terminology,** e.g. switching between n and N for the same quantity, or between “node” and “server” for the same concept.
8. **Claims with no evidence,** e.g. “this is clearly the best approach” without a measurement, citation, or argument to support it.
9. **Missing or informal citations,** e.g. “according to a blog post I found” instead of a proper reference, or reusing text from a source without paraphrasing and citing it (Section 6).
10. **Conclusion that introduces new material.** The conclusion should summarise and reflect on what came before, not present new results or arguments for the first time.
11. **Ignoring L^AT_EX warnings.** “Overfull \hbox”, undefined references, and missing citations shown as “[?]” in the compiled PDF are not cosmetic — fix them before submitting.

Common pitfall

The single most common issue is not grammar — it is *structure*: results and discussion mixed together, no clear research question, or a conclusion that repeats the abstract word for word. Fix structure first; polish sentences last.

9 Pre-submission Checklist

Run through this list before you submit. Most items take under a minute and catch the mistakes graders notice first.

Content

- ☐ The abstract states problem, approach, and key result in a few sentences.
- ☐ The introduction states the goal/research question explicitly and says what was done, in the first page.
- ☐ No heading is left without text introducing the following subsection.
- ☐ Every figure and table has a caption that stands alone, labelled axes/units, and is referenced from the body text.
- ☐ Results are reported before they are interpreted; discussion is clearly separated from results.
- ☐ Claims are backed by a measurement, a citation, or an explicit argument — not just asserted.
- ☐ Limitations/threats to validity are mentioned honestly.
- ☐ The conclusion summarises rather than introduces new material.

References

- ☐ Every non-trivial claim taken from elsewhere is cited.
- ☐ Every entry in `references.bib` that is cited actually compiles (no “[?]” in the PDF).
- ☐ Citation style is consistent throughout.

Mechanics

- ☐ The document compiles cleanly: no undefined references, no missing citations, no overfull boxes you have not checked.
- ☐ Notation and terminology are consistent from start to end.
- ☐ Spelling and grammar have been checked.
- ☐ Page/word limits (if any) are respected.
- ☐ Filenames and any submission instructions from the course have been followed.

Tip

Read the report once, start to finish, out loud or to a friend, the evening before it is due (not the minute before). You will catch things a silent read misses.

References

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] C. E. Shannon, “A mathematical theory of communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [3] A. M. Turing, “Computing machinery and intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.